

Relazione del Progetto: Escape from Musci

1. Dati del progetto

Campo	Dettaglio
Studente	Alessandro Chiloti, Federico Marku
Classe	3BTLC
Titolo del gioco	Escape from Musci
Data di consegna	31 Maggio 2026

2. Idea del gioco

Ho voluto creare un gioco di abilità e azione ambientato a scuola, prendendo spunto dalla nostra realtà di tutti i giorni. Praticamente l'idea è che c'è la verifica del Prof. Musci e il giocatore deve prima riuscire ad arrivare a scuola sani e salvi schivando il traffico, e poi deve affrontare la verifica vera e propria dentro l'aula.

La cosa originale del gioco è che si può scegliere una strategia: o si tenta il 6 studiando normalmente (ed è la via più tranquilla), oppure si punta al 10 copiando il codice Python. Se scegli di puntare al 10, il Prof. Musci ti scopre e parte una vera e propria Boss Fight finale dove devi difenderti sparando righe di codice con un fucile mentre lui ti attacca lanciando degli zeri e degli uni.

Ho pensato questo gioco per i miei compagni di classe e per chiunque conosca la fatica delle verifiche di informatica. Si vince battendo il boss (esclusivo della via del 10) o finendo i biomi del test (se si punta al 6). Si perde se finiscono le 3 vite a disposizione.

3. Analisi e progettazione

3.1 Regole del gioco

- Il giocatore parte con 3 vite sia nel gioco normale che nella boss fight.
- Nel Livello 1 bisogna evitare le macchine che passano a velocità diverse. Se vieni colpito, perdi una vita e si resetta la posizione.
- Nel Livello 2 ci sono dei blocchi di codice Python che si muovono sullo schermo. Toccandoli si perde una vita.
- Ci sono dei power-up sul percorso: il caffè dà un bonus di velocità per 5 secondi, mentre il libro di teoria dà uno scudo dorato che para il prossimo colpo.
- Nella Boss Fight il prof si muove in alto e spara proiettili binari. Il giocatore deve mirare con il mouse e sparare per togliergli gli HP. Il boss ha due fasi, nella seconda diventa più arrabbiato e veloce.

3.2 Struttura del programma

Per fare il programma ho usato la libreria **pygame** per la grafica e la gestione degli eventi, **random** per far apparire gli ostacoli in posizioni diverse e gestirne la velocità, **math** per calcolare le traiettorie dei proiettili, e **array** insieme a **pygame.mixer** per generare i suoni senza dover caricare file esterni.

Ad alto livello il codice segue questa logica (pseudocodice):

All'avvio:

Inizializza Pygame e l'audio procedurale

Mostra il MENU principale

Nel ciclo principale (60 FPS):

Legge gli input (tastiera e click del mouse)

Se lo stato è MENU:

Se clicchi "VAI A SCUOLA" -> vai a inserimento nome, poi LVL1

Se lo stato è LVL1 (Traffico):

Muovi il giocatore con WASD / Freccie

Muovi le macchine

Se c'è collisione -> perdi una vita o consuma lo scudo

Se arrivi in cima e premi SPAZIO -> vai al corridoio della scuola

Se lo stato è LVL2 (Verifica nei Biomi):

Muovi i blocchi di codice Python

Se tocchi il codice -> subisci danno

Se arrivi in cima e premi SPAZIO -> aumenti il voto

Se voto raggiunge il target (6 o 10) -> vittoria o BOSS INTRO

Se lo stato è BOSS FIGHT:

Muovi il Prof. Musci (Fase 1 o Fase 2)

Gestisci gli spari del giocatore (mira con il mouse)

Gestisci gli attacchi del Boss

Se il Boss finisce gli HP -> vittoria finale

Aggiorna la grafica (disegna personaggio, particelle, testo e barre degli HP)

Se le vite arrivano a 0 -> mostra GAME_OVER

3.3 Strutture dati importanti

- **Lista cars:** contiene dei dizionari. Ogni dizionario rappresenta una macchina con coordinata X, Y, larghezza, altezza, velocità e colore.
- **Lista lvl2 codes:** contiene i testi dei frammenti Python (es. 'while True:') con le loro posizioni e velocità di scorrimento.
- **Lista player bullets e boss attacks:** servono per tracciare tutti i colpi attivi sullo schermo, calcolando lo spostamento X e Y a ogni frame basandosi sulla trigonometria.
- **Lista particles and damage popups:** usate per gli effetti speciali visivi. Salviamo la posizione, la velocità e un timer che serve a far sparire l'effetto dopo un po' di tempo.

4. Sviluppo e uso dell'AI

4.1 Come ho lavorato

Ho lavorato da solo a questo progetto. Per prima cosa ho buttato giù l'idea di base del gioco delle macchine (Livello 1), che era la cosa più semplice da implementare per capire come funzionavano le collisioni su Pygame. Poi ho aggiunto la logica dei livelli e la transizione delle schermate, che all'inizio mi creava un po' di confusione perché il gioco si resettava male. La parte che mi ha preso più tempo è stata sicuramente la gestione della Boss Fight e la matematica per far sparare il fucile verso la posizione del mouse. Ho avuto un po' di difficoltà anche a bilanciare i danni per non rendere il gioco impossibile da finire.

4.2 Come ho usato l'AI

Ho usato l'AI Gemini per creare il gioco completamente (dalla logica iniziale alla Boss Fight) e

mi ha fornito un supporto essenziale per risolvere i bug che si sono presentati durante lo sviluppo. In particolare, l'AI mi ha aiutato a:

1. Scrivere la funzione per generare i suoni a 8-bit direttamente dal codice usando le onde sinusoidali della libreria math e la gestione dei buffer audio, evitando di dover cercare file .wav su internet.
2. Creare il sistema di particelle per le esplosioni quando il boss viene colpito o quando il giocatore perde una vita.
3. Risolvere un bug di collisione in cui il giocatore perde tutte e tre le vite istantaneamente perché il gioco continuava a rilevare il contatto con la macchina nello stesso frame.

5. Test e risultati

5.1 Casi di test

Ho eseguito diversi test manuali durante lo sviluppo per assicurarmi che le regole funzionassero a dovere:

Test	Cosa ho fatto	Cosa mi aspettavo	Cosa è successo
1	Il giocatore viene colpito da una macchina senza scudo	Perde 1 vita e parte il timer di morte temporanea	Funziona, il giocatore diventa rosso e si schiaccia
2	Il giocatore ha lo scudo attivo e tocca un ostacolo	Lo scudo si rompe ma le vite restano intatte	Funziona, lo scudo scompare ed escono particelle gialle
3	Sparo al boss nella prima fase portando gli HP a 0	Il gioco deve passare alla fase di avvio (BOSS REVIVE)	Funziona, appare la scritta di errore e il boss si rigenera
4	Clicco sul pulsante BALZA nel menu	Il gioco si deve chiudere immediatamente	Funziona correttamente senza dare errori nel terminale

5.2 Bug e problemi

C'è un piccolo problema grafico che non sono riuscito a risolvere del tutto: a volte, se ci sono

troppe particelle a schermo durante la seconda fase del boss, il testo dei proiettili del giocatore si sovrappone leggermente rendendo un po' difficile leggere cosa c'è scritto. Inoltre, per via del calcolo dei confini, se si tiene premuto contemporaneamente SU e SINISTRA nell'angolo in alto, il personaggio si blocca per un millisecondo prima di scivolare correttamente lungo il bordo. Ho deciso di lasciarlo così perché alla fine non rovina l'esperienza di gioco.

6. Conclusioni e riflessione personale

Questo progetto mi è piaciuto un sacco perché mi ha permesso di vedere come si crea un videogioco partendo da zero. Ho imparato a usare meglio i cicli e soprattutto a capire l'importanza della gestione degli stati all'interno del ciclo principale di Pygame. Prima pensavo che i giochi fossero solo immagini che si muovono, invece dietro c'è un sacco di logica matematica, specialmente per le collisioni e le traiettorie.

Riguardo all'uso dell'AI, penso di averla usata in modo intelligente: invece di farmi scrivere tutto il codice dall'inizio alla fine (cosa che mi avrebbe fatto capire poco o nulla), le chiedo piccoli pezzi di codice o spiegazioni su come implementare concetti specifici, tipo l'audio sinusoidale o i timer per i power-up. Se dovessi rifarlo da capo, forse organizzerei il codice dividendo il programma in più file .py separati (uno per il boss, uno per i livelli) per non avere un unico file gigantesco difficile da scorrere.